

**TRƯỜNG ĐẠI HỌC KHOA HỌC
KHOA CÔNG NGHỆ THÔNG TIN**



TIỂU LUẬN

PHÁT TRIỂN ỨNG DỤNG IOT

Thiết kế hệ thống giám sát nhiệt độ và độ ẩm dựa trên ESP32

**Giáo viên hướng dẫn:
Võ Viết Dũng**

**Sinh viên thực hiện :
Phạm Văn Bình Minh
Mã Sinh viên: 22t1020230**

Năm học: 2026-2027

Huế, 7/2

DANH MỤC CÁC TỪ VIẾT TẮT

Từ viết tắt	Thuật ngữ đầy đủ (Tiếng Anh)	Nghĩa tiếng Việt
IoT	Internet of Things	Internet vạn vật
ESP32	Espressif Systems Smart Connected Devices	Tên dòng vi điều khiển của hãng Espressif
GPIO	General Purpose Input/Output	Các chân ngõ vào/ra đa năng
DHT	Digital Humidity and Temperature	Cảm biến độ ẩm và nhiệt độ kỹ thuật số
Wi-Fi	Wireless Fidelity	Hệ thống kết nối mạng không dây
API	Application Programming Interface	Giao diện lập trình ứng dụng
HTTP	Hypertext Transfer Protocol	Giao thức truyền tải siêu văn bản
IDE	Integrated Development Environment	Môi trường phát triển tích hợp
VCC	Voltage at the Common Collector	Chân cấp nguồn dương (+)
GND	Ground	Chân nối đất / Cực âm (-)
SDA	Serial Data	Chân truyền dữ liệu nối tiếp
ADC	Analog-to-Digital Converter	Bộ chuyển đổi tương tự sang số
UART	Universal Asynchronous Receiver-Transmitter	Bộ truyền nhận dữ liệu nối tiếp bất đồng bộ
CPU	Central Processing Unit	Bộ vi xử lý trung tâm
SoC	System on a Chip	Hệ thống trên một vi mạch

MỤC LỤC

DANH MỤC CÁC TỪ VIẾT TẮT	2
MỤC LỤC	3
LỜI MỞ ĐẦU	5
NỘI DUNG	6
CHƯƠNG 1: GIỚI THIỆU BÀI TOÁN VÀ MỤC TIÊU ĐỀ TÀI	6
1.1 Đặt vấn đề.....	6
1.2. Mục tiêu đề tài.....	7
1.3. Đối tượng nghiên cứu.....	8
1.3.1. Vi điều khiển ESP32.....	8
1.3.2. Cảm biến nhiệt độ và độ ẩm DHT22.....	9
1.3.3. Các linh kiện hỗ trợ khác.....	9
1.4. Phương pháp nghiên cứu.....	9
1.4.1. Phương pháp nghiên cứu lý thuyết.....	9
1.4.2. Phương pháp thực nghiệm mô phỏng trên Wokwi.....	10
1.4.3. Quy trình thực hiện tổng quát.....	10
CHƯƠNG 2: THIẾT KẾ VÀ TRIỂN KHAI HỆ THỐNG MÔ PHỎNG	10
2.1. Sơ đồ nguyên lý và kết nối phần cứng.....	10
2.1.1. Kết nối cảm biến DHT22 với ESP32.....	10
2.2. Xây dựng chương trình điều khiển.....	11
2.2.1. Sơ đồ đầu nối chi tiết các linh kiện.....	11
CHƯƠNG 3: KẾT QUẢ THỰC NGHIỆM VÀ ĐÁNH GIÁ	12
3.1. Kết quả vận hành trên môi trường mô phỏng.....	12
3.1.1. Theo dõi qua Serial Monitor.....	13
3.1.2. Hiển thị dữ liệu trên Cloud ThingSpeak.....	14
3.2. Đánh giá hệ thống.....	14
3.2.1. Ưu điểm.....	14

3.2.2. Hạn chế và hướng phát triển.....	14
KẾT LUẬN VÀ KIẾN NGHỊ.....	16
1. Kết luận.....	16
2. Kiến nghị và Hướng phát triển.....	16
TÀI LIỆU THAM KHẢO.....	17
PHỤ LỤC: MÃ NGUỒN CHƯƠNG TRÌNH.....	17

LỜI MỞ ĐẦU

Trong kỷ nguyên kết nối số, Internet vạn vật (IoT) đang thay đổi mạnh mẽ cách chúng ta tương tác với môi trường xung quanh. Việc giám sát chính xác các thông số nhiệt độ và độ ẩm không chỉ quan trọng trong đời sống hằng ngày mà còn là yếu tố then chốt trong bảo quản công nghiệp, kho bãi và nông nghiệp công nghệ cao.

Nhận thấy tầm quan trọng đó, em quyết định thực hiện đề tài: “**Thiết kế hệ thống giám sát nhiệt độ và độ ẩm dựa trên ESP32**”. Tiểu luận tập trung vào việc ứng dụng vi điều khiển ESP32 và cảm biến DHT22 để thu nhập dữ liệu, sau đó truyền tải và hiển thị trực quan trên nền tảng điện toán đám mây thông qua môi trường mô phỏng Wokwi.

Dù cố gắng hoàn thiện, bài làm khó tránh khỏi những thiếu sót về mặt kiến thức và kỹ thuật. Em rất mong nhận được sự góp ý từ quý Thầy/Cô để nội dung được hoàn thiện hơn.

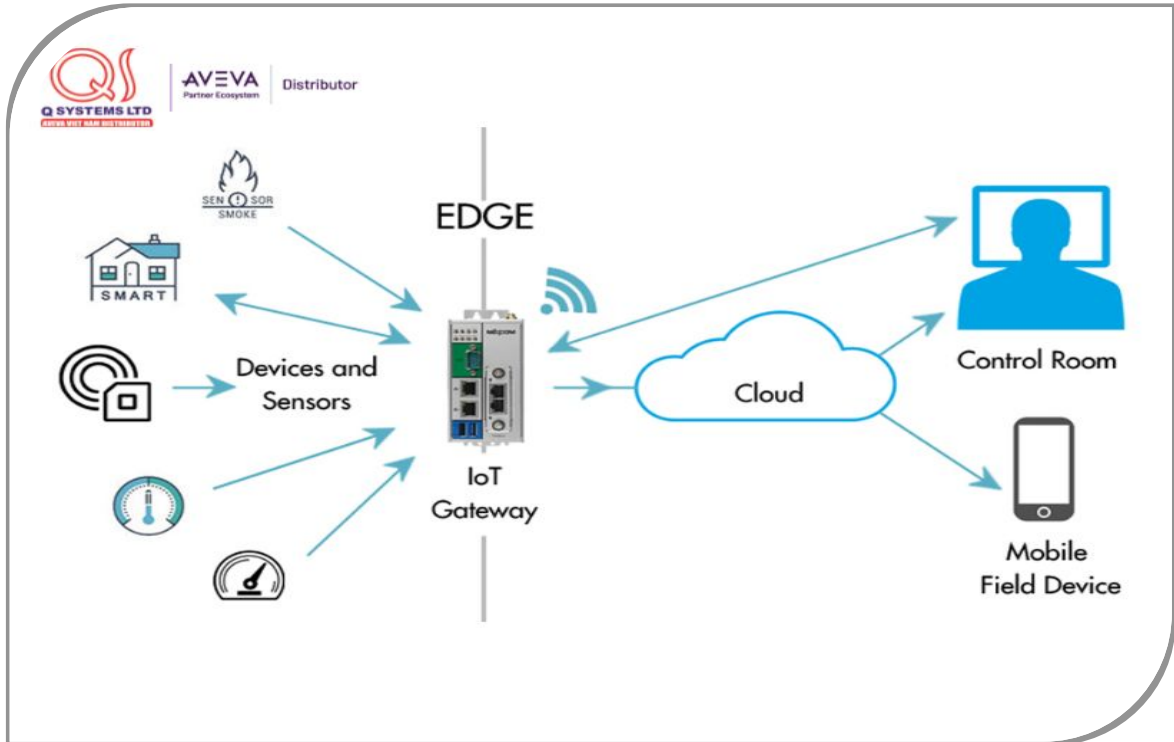
NỘI DUNG

CHƯƠNG 1: GIỚI THIỆU BÀI TOÁN VÀ MỤC TIÊU ĐỀ TÀI

1.1 Đặt vấn đề

- Trong chương trình môn học Phát triển ứng dụng IoT, việc nắm vững quy trình từ thu thập dữ liệu cảm biến đến truyền tải lên nền tảng Cloud là yêu cầu cốt lõi. Theo sự phân công của bộ môn, em thực hiện đề tài: **"Thiết kế hệ thống giám sát nhiệt độ và độ ẩm dựa trên ESP32"**.

- Hệ thống này giải quyết bài toán giám sát môi trường tự động, giúp con người có thể theo dõi dữ liệu tại những khu vực nhạy cảm (như phòng máy chủ, kho dược phẩm, nhà kính) mà không cần có mặt trực tiếp tại hiện trường.



1.1 Sơ đồ kiến trúc tổng quan hệ thống IoT

1.2. Mục tiêu đề tài

- Về kỹ thuật: Thiết kế và kết nối thành công sơ đồ mạch gồm vi điều khiển ESP32, cảm biến DHT22 trên môi trường mô phỏng Wokwi.
- Về lập trình: Xây dựng chương trình điều khiển cho ESP32 để đọc dữ liệu từ cảm biến theo chu kỳ và thực hiện giao thức HTTP để gửi dữ liệu lên máy chủ.
- Về ứng dụng Cloud: Thiết lập Channel trên ThingSpeak, cấu hình các trường (Field) dữ liệu để tiếp nhận và hiển thị kết quả dưới dạng biểu đồ trực quan thời gian thực.
- Về kiến thức: Hiểu rõ quy trình truyền tin từ thiết bị đầu cuối (End-device) qua Gateway (WiFi) đến nền tảng điện toán đám mây (Cloud).

1.3. Đối tượng nghiên cứu

1.3.1. Vi điều khiển ESP32

- ESP32 là một hệ thống trên chip (SoC - System on a Chip) mạnh mẽ, được phát triển bởi Espressif Systems. Đây là đối tượng nghiên cứu chính đóng vai trò bộ xử lý trung tâm trong hệ thống IoT này.

*2.1 Vi điều khiển
ESP32 mô phỏng
trên WokWi*



- Đặc điểm kỹ thuật:

- Bộ xử lý: Dual-core Xtensa® 32-bit LX6 với hiệu suất xử lý cao.

- Kết nối không dây: Tích hợp sẵn Wi-Fi chuẩn 802.11 b/g/n và Bluetooth (v4.2 BR/EDR và BLE), cho phép thiết bị kết nối trực tiếp với Internet mà không cần thêm module hỗ trợ.

- Hệ thống chân I/O: Sở hữu số lượng chân GPIO phong phú, hỗ trợ nhiều giao thức như ADC, DAC, UART, SPI, I2C. Trong bài toán này, chân GPIO 15 được sử dụng để đọc dữ liệu từ cảm biến

- Lý do lựa chọn: ESP32 có giá thành thấp, tiêu thụ năng lượng tối ưu và cộng đồng hỗ trợ rất lớn, đặc biệt tương thích hoàn hảo với trình mô phỏng Wokwi và nền tảng ThingSpeak.

1.3.2. Cảm biến nhiệt độ và độ ẩm DHT22

- DHT22 (hay còn gọi là AM2302) là cảm biến kỹ thuật số dùng để đo nhiệt độ và độ ẩm không khí.

- Thông số kỹ thuật chi tiết:
 - Phạm vi đo độ ẩm: Từ 0% đến 100% với độ chính xác $\pm 2\%$.
 - Phạm vi đo nhiệt độ: Từ -40°C đến 80°C với độ chính xác $\pm 0.5^{\circ}\text{C}$.
 - Tần số lấy mẫu: 0.5Hz (tương ứng với việc đọc dữ liệu 2 giây một lần).
 - Điện áp hoạt động: 3V đến 5V, phù hợp với mức nguồn 3.3V từ chân 3V3 của ESP32.



2.2 Cảm biến nhiệt độ và độ ẩm DHT22 mô phỏng trên Wokwi

- Cấu tạo và nguyên lý: Cảm biến sử dụng một điện dung đo độ ẩm và một điện trở nhiệt (thermistor) để đo nhiệt độ, sau đó chuyển đổi thành tín hiệu số (Digital) gửi qua một chân dữ liệu duy nhất (Single-bus).
- Lý do lựa chọn: So với DHT11, DHT22 có dải đo rộng và độ chính xác cao hơn, đáp ứng tốt các yêu cầu khắt khe trong việc giám sát môi trường thực tế.

1.3.3. Các linh kiện hỗ trợ khác

- **Nền tảng Cloud ThingSpeak:** Là dịch vụ lưu trữ và phân tích dữ liệu trực tuyến, cho phép biểu diễn các chỉ số thu được từ ESP32 dưới dạng biểu đồ số.

1.4. Phương pháp nghiên cứu

1.4.1. Phương pháp nghiên cứu lý thuyết

- Tìm hiểu tài liệu: Nghiên cứu các tài liệu kỹ thuật (Datasheet) của ESP32 và cảm biến DHT22 để nắm rõ thông số điện áp, sơ đồ chân và giao thức truyền tin.

- **Nghiên cứu lập trình:** Tìm hiểu ngôn ngữ lập trình C++ cho vi điều khiển trên nền tảng Arduino IDE. Đặc biệt nghiên cứu cách sử dụng thư viện DHTesp.h để giải mã tín hiệu từ cảm biến và thư viện ThingSpeak.h để truyền dữ liệu qua Internet.
- **Giao thức truyền tin:** Nghiên cứu phương thức gửi dữ liệu lên Server thông qua API Key (Application Programming Interface Key) và cấu trúc bản tin HTTP GET/POST.

1.4.2. Phương pháp thực nghiệm mô phỏng trên Wokwi

- Do điều kiện thực tế, hệ thống được xây dựng và kiểm tra tính khả thi trên trình mô phỏng **Wokwi**. Đây là phương pháp tối ưu giúp giả lập phần cứng một cách chính xác trước khi triển khai thực tế.

- **Thiết lập môi trường:** Sử dụng Wokwi để lắp ráp các linh kiện ảo. Môi trường này hỗ trợ đầy đủ ESP32 và cảm biến DHT22, cho phép kết nối dây trực quan.
- **Mô phỏng kết nối mạng:** Sử dụng WiFi ảo (Wokwi-GUEST) để kiểm tra khả năng giao tiếp giữa thiết bị và Internet.
- **Kiểm tra dữ liệu thực tế:** Thực hiện thay đổi giá trị nhiệt độ và độ ẩm trên cảm biến DHT22 trong môi trường mô phỏng để quan sát sự biến thiên của dữ liệu trên biểu đồ ThingSpeak.

1.4.3. Quy trình thực hiện tổng quát

- Quy trình được thực hiện theo các bước sau:

1. Bước 1: Thiết kế sơ đồ mạch phần cứng trên Wokwi.
2. Bước 2: Lập trình mã nguồn điều khiển dựa trên các thư viện hỗ trợ.
3. Bước 3: Cấu hình Channel trên ThingSpeak để nhận dữ liệu.
4. Bước 4: Chạy mô phỏng, theo dõi kết quả qua Serial Monitor và biểu đồ trực tuyến.
5. Bước 5: Tổng hợp dữ liệu và viết báo cáo tiểu luận

CHƯƠNG 2: THIẾT KẾ VÀ TRIỂN KHAI HỆ THỐNG MÔ PHỎNG

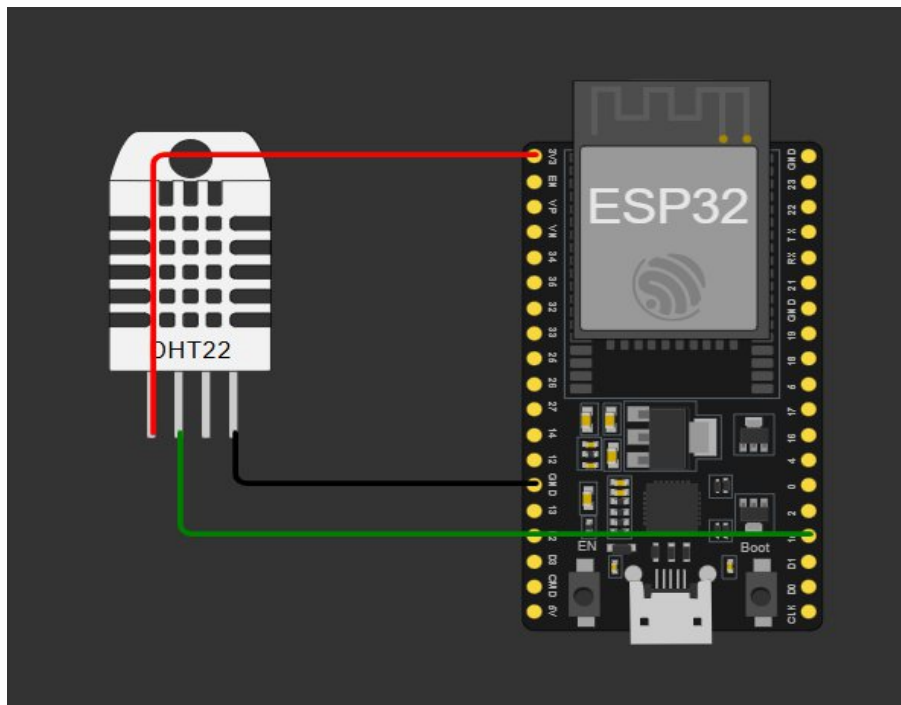
2.1. Sơ đồ nguyên lý và kết nối phần cứng

- Trong môi trường mô phỏng Wokwi, các linh kiện được kết nối với nhau thông qua hệ thống dây dẫn ảo nhưng tuân thủ nghiêm ngặt nguyên lý điện tử thực tế.

2.1.1. Kết nối cảm biến DHT22 với ESP32

- Cảm biến DHT22 được kết nối với vi điều khiển theo cấu hình 3 chân để đảm bảo việc truyền nhận dữ liệu số ổn định.

- . Chân nguồn (VCC): Nối với chân **3V3** của ESP32 để cấp nguồn 3.3V.
- . Chân dữ liệu (SDA/Data): Nối với chân **GPIO 15 (D15)** của ESP32. Đây là chân được cấu hình trong mã nguồn để đọc xung tín hiệu từ cảm biến.
- . Chân đất (GND): Nối với chân **GND** của ESP32 để tạo mạch kín.



. 2.1 Sơ đồ kết nối ESP32 với cảm biến DHT22 trên Wokwi

2.2. Xây dựng chương trình điều khiển

- Phần mềm điều khiển được viết dựa trên nền tảng Arduino với các khối chức năng chính:

1. Khởi khởi tạo (Setup): Thiết lập kết nối WiFi ảo Wokwi-GUEST và cấu hình các chân I/O.
2. Khối đọc dữ liệu: Sử dụng thư viện DHTesp để lấy giá trị nhiệt độ và độ ẩm từ chân 15 mỗi 15-20 giây.
3. Khối truyền dữ liệu: Sử dụng giao thức HTTP để gửi dữ liệu lên Server ThingSpeak. Mỗi lần gửi thành công, mã phản hồi HTTP 200 sẽ được trả về và hiển thị trên Serial Monitor.

2.2.1. Sơ đồ đấu nối chi tiết các linh kiện

- Để hệ thống vận hành ổn định và chính xác, việc kết nối các chân tín hiệu giữa cảm biến DHT22 và vi điều khiển ESP32 được thực hiện theo bảng thông số dưới đây:

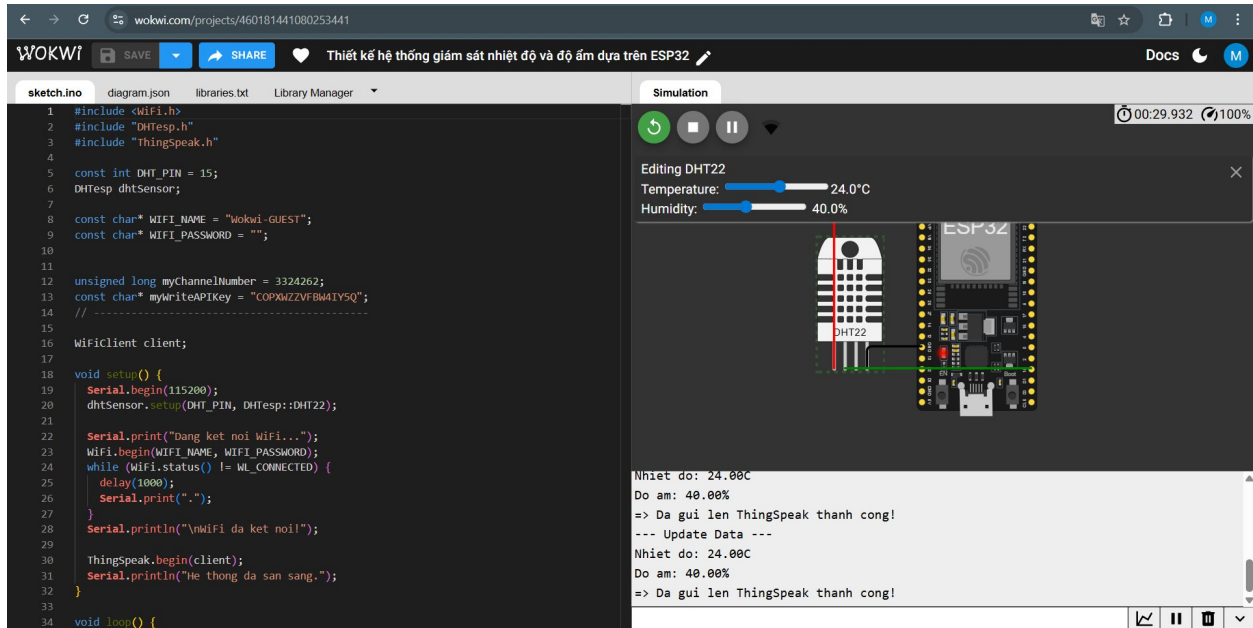
STT	Tên linh kiện	Chân linh kiện	Chân kết nối trên ESP32
1	Cảm biến DHT22	VCC	3V3
2	Cảm biến DHT22	SDA (Data)	GPIO 15
3	Cảm biến DHT22	NC	Không nối
4	Cảm biến DHT22	GND	GND

Bảng 2.1 Danh mục kết nối chân giữa DHT22 và ESP32

CHƯƠNG 3: KẾT QUẢ THỰC NGHIỆM VÀ ĐÁNH GIÁ

3.1. Kết quả vận hành trên môi trường mô phỏng

Sau khi hoàn thiện việc lắp ráp và lập trình, hệ thống được vận hành thử nghiệm trên Wokwi. Kết quả thu được cho thấy sự ổn định và chính xác trong quá trình truyền tin.



3.1 Kết quả kết nối thành công hiển thị trên Serial Monitor.

3.1.1. Theo dõi qua Serial Monitor

Khung Serial Monitor đóng vai trò là giao diện giám sát tại chỗ. Kết quả hiển thị bao gồm:

- **Trạng thái kết nối:** Hệ thống báo kết nối thành công với mạng WiFi ảo Wokwi-GUEST.
- **Dữ liệu đo đạc:** Các giá trị nhiệt độ và độ ẩm được cập nhật liên tục sau mỗi 15-20 giây. Ví dụ: *Nhiệt độ: 24.00°C, Độ ẩm: 40.00%*.
- **Trạng thái gửi tin:** Hiển thị dòng thông báo "*Da gui len ThingSpeak thanh cong!*" kèm mã phản hồi HTTP 200, xác nhận gói tin đã được máy chủ chấp nhận.

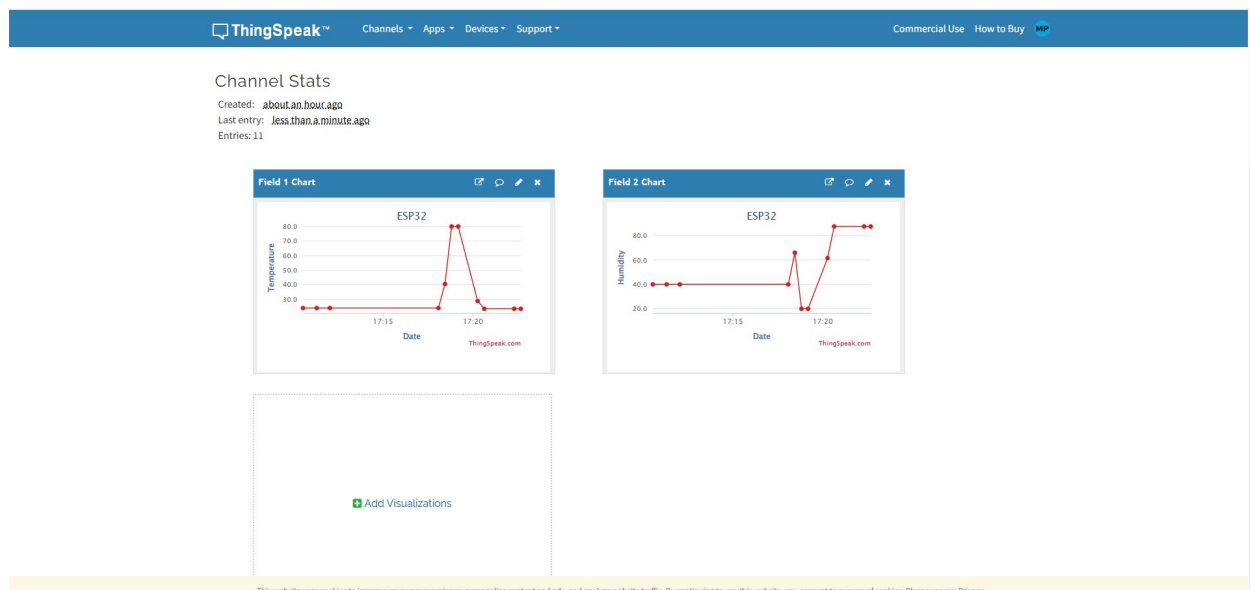
```
Nhiệt độ: 24.00C
Độ ẩm: 40.00%
=> Đã gửi lên ThingSpeak thành công!
--- Update Data ---
Nhiệt độ: 24.00C
Độ ẩm: 40.00%
=> Đã gửi lên ThingSpeak thành công!
```

3.2 Dữ liệu vận hành hệ thống hiển thị qua Serial Monitor.

3.1.2. Hiển thị dữ liệu trên Cloud ThingSpeak

Đây là kết quả quan trọng nhất của hệ thống IoT. Dữ liệu từ ESP32 được trực quan hóa dưới dạng biểu đồ thời gian thực:

- ❖ **Field 1 Chart:** Hiển thị biến thiên của nhiệt độ theo thời gian.
- ❖ **Field 2 Chart:** Hiển thị biến thiên của độ ẩm theo thời gian.
- ❖ **Đặc điểm:** Các điểm dữ liệu được nối với nhau tạo thành đường biểu đồ giúp người dùng dễ dàng nhận thấy xu hướng thay đổi của môi trường.



3.3 Biểu đồ giám sát nhiệt độ và độ ẩm trên Cloud ThingSpeak

3.2. Đánh giá hệ thống

3.2.1. Ưu điểm

- Tính tức thời: Dữ liệu được cập nhật nhanh chóng, độ trễ thấp (khoảng dưới 2 giây từ khi đọc đến khi hiện trên web).
- Tính trực quan: Biểu đồ trên ThingSpeak giúp người dùng theo dõi dễ dàng mà không cần kiến thức kỹ thuật sâu.

3.2.2. Hạn chế và hướng phát triển

- ❗ Hạn chế: Hệ thống hiện tại mới chỉ dừng lại ở mức giám sát, chưa có chức năng điều khiển thiết bị ngoại vi (như tự động bật quạt khi nóng).
- ✓ Hướng phát triển: Tích hợp thêm các nút nhấn điều khiển trên Dashboard của ThingSpeak và bổ sung module cảnh báo qua Email hoặc tin nhắn điện thoại khi nhiệt độ vượt ngưỡng cho phép.

KẾT LUẬN VÀ KIẾN NGHỊ

1. Kết luận

- Sau một thời gian nghiên cứu và triển khai thực nghiệm mô phỏng, đề tài "**Giám sát nhiệt độ và độ ẩm thời gian thực sử dụng ESP32 và Cloud ThingSpeak**" đã hoàn thành các mục tiêu đề ra ban đầu. Những kết quả đạt được cụ thể như sau:

- Về mặt lý thuyết: Đã nghiên cứu và làm chủ được cách thức hoạt động của vi điều khiển ESP32, cảm biến DHT22 cũng như quy thức truyền tin HTTP lên nền tảng điện toán đám mây.
- Về mặt thiết kế: Xây dựng thành công sơ đồ nguyên lý trên môi trường Wokwi với đầy đủ các linh kiện hỗ trợ như điện trở, đảm bảo tính đúng đắn về mặt kỹ thuật điện tử.
- Về mặt vận hành: Hệ thống đã thực hiện tốt việc đọc dữ liệu môi trường và truyền tải ổn định lên Server ThingSpeak. Dữ liệu được hiển thị dưới dạng biểu đồ trực quan, giúp người dùng dễ dàng theo dõi từ xa.
- Tính ứng dụng: Mặc dù chỉ là mô phỏng, nhưng hệ thống cho thấy tiềm năng rất lớn trong việc áp dụng vào thực tế tại các kho lưu trữ, nhà kính hoặc căn hộ thông minh với chi phí thấp và hiệu quả cao.

2. Kiến nghị và Hướng phát triển

- Để hệ thống hoàn thiện hơn và có thể đưa vào sử dụng thực tế rộng rãi, nhóm thực hiện xin đưa ra một số kiến nghị phát triển như sau:

- Nâng cấp phần cứng: Trong tương lai, có thể thay thế mô phỏng bằng thiết bị thực tế để đánh giá sai số của cảm biến trong các điều kiện môi trường khắc nghiệt khác nhau.
- Tăng cường bảo mật: Nghiên cứu các giao thức truyền tin bảo mật hơn như HTTPS hoặc MQTT với cơ chế mã hóa để bảo vệ dữ liệu người dùng.
- Tích hợp điều khiển: Bổ sung thêm các rơ-le (Relay) để tự động điều chỉnh thiết bị (ví dụ: tự bật máy phun sương khi độ ẩm thấp) tạo thành một hệ thống IoT khép kín.
- Ứng dụng di động: Phát triển ứng dụng trên điện thoại thông minh để nhận thông báo đẩy (Push Notification) khi nhiệt độ vượt quá ngưỡng an toàn

TÀI LIỆU THAM KHẢO

1. **Võ Viết Dũng** (2026), *Bài giảng Phát triển ứng dụng IoT*, Khoa Công nghệ thông tin, Trường Đại học Khoa học - Đại học Huế. Địa chỉ : https://github.com/vvdung-husc/2025-2026.2.TIN4024.002/blob/main/Team_DHT_OLED.md
2. **MathWorks** (2026), *ThingSpeak Documentation - Collect Data in the Cloud*, [Trực tuyến]. Địa chỉ: <https://www.mathworks.com/help/thingspeak/>
3. **Espressif Systems** (2023), *ESP32 Series Datasheet v4.1*, [Trực tuyến]. Địa chỉ: https://www.espressif.com/sites/default/files/documentation/esp32_datasheet_en.pdf
4. **Wokwi Documentation** (2026), *ESP32 Simulator and DHT22 Sensor Integration Guide*, [Trực tuyến]. Địa chỉ: <https://docs.wokwi.com/parts/wokwi-dht22>

PHỤ LỤC: MÃ NGUỒN CHƯƠNG TRÌNH

```
#include <WiFi.h>
#include "DHTesp.h"
#include "ThingSpeak.h"

const int DHT_PIN = 15;
DHTesp dhtSensor;

const char* WIFI_NAME = "Wokwi-GUEST";
const char* WIFI_PASSWORD = "";

unsigned long myChannelNumber = 3324262;
const char* myWriteAPIKey = "COPXWZZVFBW4IY5Q";
// -----

WiFiClient client;

void setup() {
    Serial.begin(115200);
    dhtSensor.setup(DHT_PIN, DHTesp::DHT22);

    Serial.print("Dang ket noi WiFi...");
    WiFi.begin(WIFI_NAME, WIFI_PASSWORD);
    while (WiFi.status() != WL_CONNECTED) {
        delay(1000);
        Serial.print(".");
    }
    Serial.println("\nWiFi da ket noi!");

    ThingSpeak.begin(client);
    Serial.println("He thong da san sang.");
}

void loop() {
    TempAndHumidity data = dhtSensor.getTempAndHumidity();
```



```
float temp = data.temperature;
float humid = data.humidity;

if (isnan(temp) || isnan(humid)) {
    Serial.println("Loi: Khong doc duoc du lieu tu cam bien!");
} else {
    Serial.println("--- Update Data ---");
    Serial.print("Nhiet do: "); Serial.print(temp);
Serial.println("C");
    Serial.print("Do am: "); Serial.print(humid); Serial.println("%");

    ThingSpeak.setField(1, temp);
    ThingSpeak.setField(2, humid);

    int x = ThingSpeak.writeFields(myChannelNumber, myWriteAPIKey);

    if(x == 200){
        Serial.println("=> Da gui len ThingSpeak thanh cong!");
    } else {
        Serial.println("=> Loi gui du lieu. Ma loi HTTP: " + String(x));
    }
}

delay(20000);
}
```

